DEPARTMENT OF CSE, CS&IT, AI&DS **2400031415 D.Keerthana**

COURSE CODE: 24SDCS02 / 24SDCS02E / 24SDCS02P / 24SDCS02L

FULL STACK APPLICATION DEVELOPMENT

Date of the Session: _/_/_____ Time of The Session: ___to_____

#SKILL - 4  Spring Dependency Injection – Constructor & Setter Injection

Prerequisites:

- Basic knowledge of Java
- General Idea on Spring Framework
- Basic Idea on DI & IoC

A training institute wants to automate the management of student information in a Spring application. The system should inject details such as studentId, name, course, and academic year into objects without manually assigning values. This Skill demonstrates Constructor Injection and Setter Injection using both XML and Annotation configurations.

Tasks:

1. Create a Java class named Student with fields: studentId, name, course, year.
2. Create a Constructor that accepts all fields and assigns values.
3. Create Setter methods for at least two fields (example: course, year).
4. Configure the Student bean using:

a. XML Configuration - The student should take:

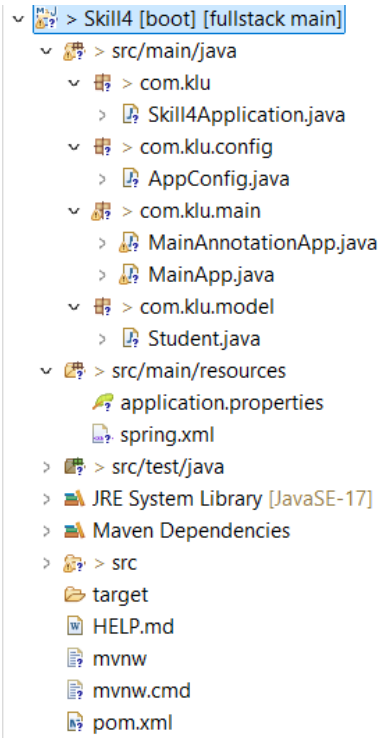
- a POJO class
- an XML configuration file
- a main class to load the container

b. Annotation Configuration - The student should take:

- a POJO class
- a Java class with necessary annotations
- a main class to load the container

5. Create a Spring configuration file (XML or Java-based).
6. Write a main class to load the Spring IoC container.
7. Retrieve the Student bean and print the injected values.
8. Push the Spring (core) project to GitHub.

**Project Structure:**




**Skill4Application.java**

```
package com.klu;

import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class Skill4Application {

	public static void main(String[] args) {

		SpringApplication.run(Skill4Application.class, args);

	}

}
```

**AppConfig.java**

```
package com.klu.config;

import org.springframework.context.annotation.ComponentScan;

import org.springframework.context.annotation.Configuration;

@Configuration
```

```
@ComponentScan(basePackages = "com.example")
public class AppConfig {
}
```

**MainAnnotationApllicationApp.java**

```
package com.klu.main;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import com.klu.config.AppConfig;
import com.klu.model.Student;
public class MainAnnotationApp {
    public static void main(String[] args) {
        ApplicationContext context =
                new AnnotationConfigApplicationContext(AppConfig.class);
        Student student = context.getBean(Student.class);
        student.display();
    }
}
```

**MainApp.java**

```
package com.klu.main;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.klu.model.Student;
public class MainApp {
    public static void main(String[] args) {
        ApplicationContext context =
                new ClassPathXmlApplicationContext("spring.xml");
        Student student = (Student) context.getBean("student");
        student.display();
    }  }
```

**Student.java**

```
package com.klu.model;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;
@Component
public class Student {
   private int studentId;
   private String name;
   private String course;
   private int year;
   public Student(@Value("101") int studentId,
               @Value("Vasavi") String name,
               @Value("Spring Framework") String course,
               @Value("2") int year) {
      this.studentId = studentId;
      this.name = name;
      this.course = course;
      this.year = year;
   }
   public void setCourse(String course) {
      this.course = course;
   }
   public void setYear(int year) {
      this.year = year;
   }
   public void display() {
      System.out.println("Student ID : " + studentId);
      System.out.println("Name       : " + name);
      System.out.println("Course     : " + course);
      System.out.println("Year       : " + year);
   }  }
```

**Spring.xml**

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="student" class="com.klu.model.Student">
     <constructor-arg value="101"/>
     <constructor-arg value="Vasavi"/>
     <constructor-arg value="FSAD"/>
     <constructor-arg value="2"/>
     <property name="course" value="FSAD"/>
     <property name="year" value="2"/>
  </bean>
</beans>
```

**pom.xml**

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
        <modelVersion>4.0.0</modelVersion>
        <parent>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-starter-parent</artifactId>
                <version>4.0.1</version>
                <relativePath/> <!-- lookup parent from repository -->
        </parent>
        <groupId>com.klu</groupId>
        <artifactId>spring-di</artifactId>
```

```
<version>0.0.1-SNAPSHOT</version>
    <name>Skill4</name>
    <description>Demo project for Spring Boot</description>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
    <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
    <properties>
        <java.version>17</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies
```

```
    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

Output:

```
<terminated> Skill4 - MainApp
Student ID  :  101
Name        :  Keerthan
Course      :  FSAD
Year        :  2
```

**VIVA QUESTIONS:**

1. What is Dependency Injection (DI) in Spring?
2. What is the difference between Constructor DI and Setter DI?
3. What is the purpose of the IoC container?
4. Why is DI preferred over manual object creation?
5. Can Constructor Injection and Setter Injection be used together?

**1Ans:** Dependency Injection is a design pattern where Spring provides required objects to a class instead of the class creating them itself.

**2Ans: Constructor DI:** Dependencies are provided through the constructor (mandatory dependencies).

**Setter DI:** Dependencies are provided through setter methods (optional dependencies).

**3Ans:** The IoC container manages object creation, dependency injection, configuration, and lifecycle of Spring beans.

**4Ans:** DI promotes loose coupling, better testability, easier maintenance, and flexibility in changing implementations.

**5Ans:** Yes, both can be used together in the same class — constructor for required dependencies and setter for optional ones.

**(For Evaluator's use only)**

| Comment of the Evaluator (if Any) | Evaluator's Observation<br><br>Marks Secured:____out of _<br><br>EMP ID & Full Name of the Evaluator:<br><br>Signature of the Evaluator Date of Evaluation: |
|---|---|